

SUQL: Conversational Search over Structured and Unstructured Data

Objective, Method, Pipeline, and Extensions for Heterogeneous Retrieval

Based on Liu et al., “SUQL: Conversational Search over Structured and Unstructured Data with Large Language Models”.

Core Problem

Many real knowledge bases are hybrid:

- **Structured fields:** IDs, dates, ratings, prices, categories, foreign keys.
- **Unstructured fields:** reviews, passages, PDF chunks, comments, descriptions.

Question example:

Find an Italian restaurant in San Francisco with a romantic atmosphere.

This requires both:

- exact structured filtering: `cuisine = Italian, location = San Francisco;`
- semantic text reasoning over reviews: “is the atmosphere romantic?”

Standard RAG alone is weak here: embedding search can retrieve semantically similar rows but fail exact constraints, sorting, aggregation, and joins.

Objective of the SUQL Approach

Goal: build a conversational QA/search system over hybrid data while preserving database-style precision.

Main idea:

- Keep structured data in relational form.
- Add LLM-powered text operators inside SQL.
- Translate user questions into executable hybrid queries.

SUQL = SQL + free-text primitives

`ANSWER(text, question)` and `SUMMARY(text)`

This gives a formal, inspectable, optimizable query representation instead of a purely prompt-based RAG chain.

SUQL Language: SQL with Text Semantics

Example natural-language query:

Find a family-friendly restaurant that serves burgers in D.C.

```
SELECT *, summary(reviews),  
        answer(reviews, 'is this restaurant family-friendly?')  
FROM restaurants  
WHERE answer(reviews,  
            'do you find this restaurant family-friendly?') = 'Yes'  
      AND answer(popular_dishes,  
            'does this restaurant serve burgers?') = 'Yes'  
      AND location = 'D.C.'  
LIMIT 1;
```

Important point: ANSWER is usable inside SELECT, WHERE, ORDER BY, nested queries, and joins, because it behaves like a SQL expression.

Input, Intermediate Output, Final Output

Input data

- Relational tables
- Text columns
- Table schema
- Optional dialogue history
- User utterance

Intermediate output

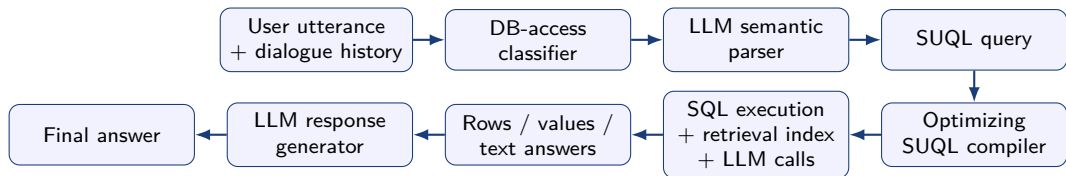
- Parsed SUQL query
- Candidate rows from SQL filters
- Candidate rows from dense retrieval
- LLM-verified text predicates
- Executed table result

Final output

- Conversational answer
- Row/value answer
- Summary of evidence
- In an engineering system: CSV/JSON + citations

For production: the final answer should not be only generated text. It should include selected rows, evidence spans, table/cell provenance, and confidence signals.

Whole Pipeline of the Original SUQL System



Key design choice: the entire query is produced before execution, so the system can optimize execution instead of blindly calling the LLM on every row.

Methods Implemented in the Paper

① Semantic parsing with in-context learning

LLM maps user utterance + schema + examples to SUQL.

② Hybrid query language

SQL handles relational logic; ANSWER/SUMMARY handle text semantics.

③ Optimizing compiler

Reorders predicates, pushes cheap filters first, uses lazy evaluation under LIMIT.

④ Dense retrieval as an index for text predicates

Embeddings retrieve likely candidate rows before expensive LLM verification.

⑤ ENUM normalization

LLM maps user terms such as “coffee” to allowed enum values such as “coffee & tea” or “cafe”.

⑥ Final response generation

LLM verbalizes the executed result and is instructed not to hallucinate when no rows are found.

Compiler Optimization Logic

Naive execution is too expensive:

```
answer(reviews, 'is parking easy?') = 'Yes'
```

would require one LLM call per row.

Optimized execution:

- 1 Apply deterministic SQL filters first: city, price, rating, year, category.
- 2 Use dense retrieval over text columns to get likely candidates.
- 3 Verify candidate rows using the original ANSWER predicate.
- 4 Stop early if LIMIT is satisfied.

Why verification is necessary: retrieval may find a review mentioning “parking”, but the review may say parking is bad. Similarity is not truth.

Why This Is Better Than Linearized RAG

Capability	Linearized RAG	SUQL-style query execution
Exact filtering	Weak	Strong
Numerical comparisons	Fragile	Native SQL
Aggregation/sorting	Hard	Native SQL
Textual constraints	Natural	Natural via ANSWER
Interpretability	Low	Inspectable query
Optimization	Mostly retrieval-time	Query-plan level
Joins	Usually lost	Possible through SQL

Main tradeoff: SUQL is more faithful and controllable, but it depends on good schema design, robust text predicates, and reliable semantic parsing.

Using SUQL for Heterogeneous Data Retrieval

SUQL does not solve raw document ingestion by itself.

It assumes hybrid data has already been converted into a queryable schema.

Adapted architecture for PDFs, tables, CSVs, and semi-structured data:

- 1 Parse PDFs, OCR if needed, extract tables and text blocks.
- 2 Normalize entities, rows, columns, units, dates, and IDs.
- 3 Store structured fields in relational tables.
- 4 Store passages/chunks as text columns or linked evidence tables.
- 5 Build embeddings over text evidence with provenance metadata.
- 6 Parse user question into SUQL.
- 7 Execute SQL + retrieval + LLM verification.
- 8 Return rows/values plus source citations.

Example: Movie Reviews + Movie Metadata

Data:

- movies: title, year, director, duration, genre, rating.
- reviews: movie_id, review_text, source, page/span provenance.

User question:

What are the 3 most interesting movies released in 1998?

```
SELECT m.title, m.year, m.director,  
       summary(r.review_text) AS review_summary  
FROM movies m  
JOIN reviews r ON r.movie_id = m.movie_id  
WHERE m.year = 1998  
      AND answer(r.review_text,  
                'do reviewers describe this movie as interesting?') = 'Yes'  
ORDER BY m.rating DESC  
LIMIT 3;
```

Output: a CSV/JSON table with selected movies, summaries, and evidence pointers.

Dropping Dialogue Memory: Stateless SUQL

The original paper includes dialogue state tracking: user turns, agent turns, follow-ups, and previous search results.

For your heterogeneous retrieval problem, remove this layer if no chat memory is needed.

Stateless pipeline:

question + schema + task instructions → SUQL → execution → table answer

Implementation updates:

- Remove database-access classifier unless all queries are retrieval queries.
- Remove dialogue history from the semantic-parser prompt.
- Do not resolve pronouns like “it”, “their”, “another one” from prior turns.
- Make context explicit through parameters: table names, filters, document collection, output format.
- Prefer deterministic output: CSV/JSON rows before natural-language explanation.

Adding Several Tables: Joins and Schema Linking

SUQL can naturally support multi-table questions if the parser sees the schema and relationships.

Example schema:

```
movies(movie_id, title, year, director_id, duration, rating)
directors(director_id, name, nationality)
reviews(review_id, movie_id, review_text, source_doc, page)
awards(movie_id, award_name, award_year)
```

Example query:

```
SELECT m.title, d.name, a.award_name,
       answer(r.review_text, 'why is this movie praised?') AS reason
FROM movies m
JOIN directors d ON m.director_id = d.director_id
JOIN reviews r ON r.movie_id = m.movie_id
LEFT JOIN awards a ON a.movie_id = m.movie_id
WHERE d.nationality = 'French'
      AND m.year BETWEEN 1990 AND 2000
      AND answer(r.review_text, 'is the acting praised?') = 'Yes'
LIMIT 5;
```

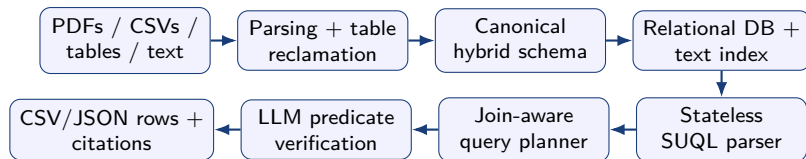
What Must Be Added for Reliable Multi-Table SUQL

The paper demonstrates the idea, but production multi-table retrieval needs extra machinery.

- **Schema linking:** map user mentions to tables, columns, enum values, and join paths.
- **Join-path selection:** choose valid paths through foreign keys instead of hallucinated joins.
- **Query validation:** parse SQL AST, check tables/columns/types before execution.
- **Cost control:** limit LLM predicates after joins to avoid Cartesian explosion.
- **Evidence model:** keep provenance at row, cell, chunk, page, and document level.
- **Confidence scoring:** combine parser confidence, retrieval score, predicate verification, and evidence coverage.

Practical rule: execute joins and deterministic filters before LLM text predicates whenever possible.

Updated Pipeline for Your Project



Key change from the paper: replace conversational dialogue tracking with a stateless, join-aware, provenance-preserving execution engine.

Main Failure Modes and Engineering Controls

Failure mode	Control
Wrong SUQL generated	SQL AST validation, constrained decoding, self-correction, schema linking
Wrong enum/value mapping	explicit dictionaries, embedding lookup, LLM classifier with logging
False positive text predicate	retrieve candidates, then verify with strict entailment-style prompt
Excessive LLM calls	predicate pushdown, LIMIT-aware lazy evaluation, caching
Noisy PDF/table extraction	table reclamation, normalization, provenance, human-checkable traces
Unsupported multi-table joins	foreign-key graph, join templates, query-plan validation
Hallucinated final answer	answer only from executed rows and evidence spans

Takeaway

SUQL is best viewed as a hybrid query execution layer.

It is useful when:

- some constraints are exact and structured;
- some constraints require semantic interpretation of text;
- the answer should be rows, values, summaries, or citations rather than free-form generation only.

For heterogeneous data retrieval, the strongest version is:

**document ingestion + schema normalization + SUQL-style
hybrid execution + provenance-grounded output**

Dropping dialogue memory simplifies the system. Adding multi-table joins makes it more useful but requires schema linking, join planning, and strict validation.